Isaiah Cordon

Dataset

Dataset source: https://huggingface.co/datasets/lmsys/chatbot_arena_conversations

I selected five LLMs for the classification task:

- ChatGLM-6B
- Claude-v1
- GPT-4
- Koala-13B
- Vicuna-13B

I used 15,000 examples in total, with 3,000 examples from each model. Using the same number of examples for every model helped me prevent the classification task from being dominated by one model simply because it had more examples.

Each example included:

- LLM_name: The name of the model that generated the response.
- LLM_Input: The prompt or input provided to the model.
- LLM_output: The response generated by the model.

The LLM name was used as the target label that the CNN and RNN models were trained to predict.

In depth Data Analysis

Before training the models, I examined the text data to look for differences between the five LLMs. Exploring response length and word count to understand whether the models tended to produce responses of different lengths.

The average word counts:

LLM_name	LLM_Input mean	LLM_output mean	Response_Length mean	Word_Count mean
chatglm-6b			842.608	143.533
claude-v1			1041.42	170.108
gpt-4			947.979	148.318
koala-13b			878.43	144.435
vicuna-13b			799.917	133.254

Claude-v1 had the highest average word count, at approximately 171 words per response. Vicuna-13B had the lowest, at approximately 133 words per response. GPT-4 was in the middle, averaging approximately 149 words per response.

These findings suggest that response length varies across the models. However, the results do not concretely establish that word count alone can reliably help identify which model generated a response.

I also calculated descriptive features such as response length, word count, and sentence count. These features were used for analysis, but they were not directly provided as input features to the CNN and RNN models. Instead, the models learned from the tokenized text itself.

Methodology

How I prepared the data

I prepared the dataset for text classification by assigning a label to each LLM:

Label	LLM
0	ChatGLM-6B
1	Claude-v1
2	GPT-4
3	Koala-13B
4	Vicuna-13B

During training, I also used a validation split of 20% of the training data. This gave approximately 9,600 examples for fitting the model and 2,400 examples for validation. The validation set was used to monitor performance during training, while the separate test set was used to evaluate the models after training.

I converted the text into numerical sequences using tokenization and padded the sequences to a maximum length of 200 tokens. This allowed the models to process text inputs in a consistent format.

Convolutional Neural Network

CNN was used as one approach to classify the text responses.

My implementation included:

Embedding layer: Converted tokens into numerical vectors with 128 dimensions.

Convolutional layer: Used 128 filters with a kernel size of 5 and ReLU activation to identify patterns in the text.

Global max pooling layer: Selected the strongest detected features from the sequence.

Dense layer: Used 64 neurons with ReLU activation to learn additional patterns.

Output layer: Used five output neurons with softmax activation to predict the probability of each LLM class.

The CNN was then rained for five epochs with a batch size of 32.

Recurrent Neural Network

I used an RNN for the same five-class classification task. The purpose was to compare its performance with the CNN and determine whether sequential processing was more effective for identifying the LLM from the text.

The RNN experiments used the same three input configurations as the CNN. The models were trained for five epochs, and I monitored training and validation accuracy during training.

Configurations

I conducted six experiments by combining two model types with three input configurations.

Experiment	Model	Input text
1	CNN	Prompt only
2	CNN	Response only
3	CNN	Prompt and response
4	RNN	Prompt only
5	RNN	Response only
6	RNN	Prompt and response

For the prompt only experiments, the model received the user's input. For the response only experiments, the model received the LLM's generated response. For the combined experiments, the prompt and response were combined into one text input.

Evaluation Metrics

Because the dataset contained five equally represented classes, the expected accuracy from random guessing was approximately 20%.

Results

Overall Test Accuracy

The final test accuracies:

Model	Input configuration	Test accuracy
CNN	Prompt only	44.2%
CNN	Response only	64.0%
CNN	Prompt and response	62.3%
RNN	Prompt only	20.3%
RNN	Response only	33.3%
RNN	Prompt and response	36.2%

The response-only CNN achieved the highest test accuracy at 64.0%. The combined-text CNN followed at 62.3%, while the prompt-only CNN achieved 44.2%.

The RNN achieved lower test accuracy in all three configurations. Its highest result was 36.2% when both the prompt and response were provided. The response-only RNN achieved 33.3%, and the prompt-only RNN achieved 20.3%.

All experiments except the prompt-only RNN performed above the expected 20% random-guessing baseline. And the prompt-only RNN was only slightly above that baseline.

Research Question 1: Can We Identify Which LLM Generated a Response?

The results suggest that LLM identification is possible to some extent. The response only CNN performed best, achieving 64.0% test accuracy. The response only RNN achieved 33.3%. Since both performed above the baseline of 20%, the text contained patterns that helped distinguish between the models. However, neither models classified every response correctly.

Research Question 2: How Does the Type of Input Text Affect Identification?

The CNN performed best using the response alone, achieving 64.0% accuracy compared with 44.2% using prompts alone. Combining the prompt and response slightly reduced its accuracy to 62.3%.

The RNN performed best using the combined prompt and response, achieving 36.2%. This was a small improvement over its response only accuracy of 33.3%.

Overall, the response alone worked best for the CNN, but combining the prompt and response worked best for the RNN.

Training and Validation

Model	Input	Final training accuracy	Best validation accuracy
CNN	Prompt only	83.5%	43.1%
CNN	Response only	98.1%	65.5%
CNN	Prompt + response	97.3%	65.0%
RNN	Prompt only	21.9%	22.2%
RNN	Response only	53.3%	33.8%
RNN	Prompt + response	58.65%	37.21%

The CNNs achieved high training accuracy but much lower validation accuracy, suggesting overfitting. The prompt only RNN struggled to learn useful patterns, with accuracy close to the random baseline.

The combined text RNN's validation accuracy peaked at 37.21% in Epoch 2 before declining to 35.29% in Epoch 5. Its validation loss also increased after Epoch 2, suggesting that it began to overfit. Its final test accuracy was 36.2%.

Lessons I learned

The CNN outperformed the RNN in all three input configurations. The response only CNN achieved the highest accuracy, meaning that generated responses contained useful patterns for identifying LLMs. However, the CNNs' large gaps between training and validation accuracy means that they did not generalize equally well to unseen data. The RNNs performed less accurately, especially when using prompts alone.

This project showed that CNNs and RNNs can identify LLMs from text with varying levels of success. The response only CNN achieved the highest test accuracy at 64.0%, while the combined text RNN achieved the best RNN accuracy at 36.2%.

The findings suggest that generated responses are useful for identifying LLMs, but classification is not always reliable. The results also emphasize the importance of reducing overfitting and evaluating models on unseen data.